

Ch 05: Value Returning Functions, Python Modules

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Value-Returning Functions

Value-Returning Function

A value-returning function is a function that returns a value back to the caller. Python, as well as most other programming languages, provide a library of prewritten functions that perform commonly needed tasks.

```
def value_returning_function(parameter1, parameter2, ...):  
    calculation1  
    calculation2  
    return result
```

```
r = value_returning_function(p1, p2, ...)
```

- **void function:** group of statements within a program for performing a specific task
 - Call function when you need to perform the task
 - The task is done as a *side-effect* of calling a void function
- **Value-returning function:** In contrast a value-returning function calculates a result from input and returns the result from calling the function
 - Value returned to the caller that invoked the function



Standard Library Functions and the `import` Statement

- **Standard library:** library of pre-written functions that comes with a programming language like Python
 - **Library functions** perform tasks that programmers commonly need
 - Example: `print()`, `input()`, `range()`
 - Viewed by programmers as a “black box”
- Some library functions built into Python interpreter
 - To use, just call the functions
- **Modules:** collections of functions in standard library
 - Help organize library functions not built into the interpreter
 - Module **namespaces**
- To call a function stored in a module, need to write an `import` statement
 - Written at the **top** of the program
 - Format: `import module_name`



Figure 1: A library function viewed as a black box (Gaddis, [2021](#), pg.273)



Generating Random Numbers (1 of 2)

- Random numbers are useful in a lot of programming tasks
- **random** module: includes library functions for working with random numbers
- **Dot notation** notation for calling a function in a module namespace
 - Format:
`module_name.function_name()`
- **randint() function**: generates a random number in the range provide by the arguments
 - Returns the random number ot part of program that called the function
 - Returned integer can be used anywhere that an integer can
 - You can experiment with the function in interactive mode

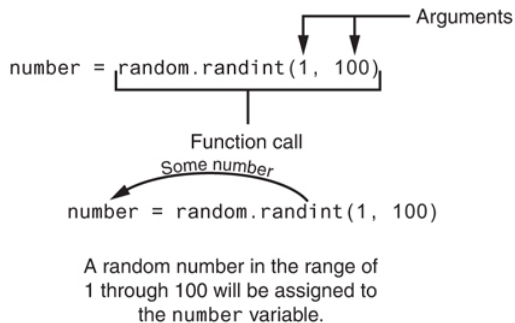


Figure 2: The random function returns a random int in asked for range (Gaddis, 2021, pg.274)



Generating Random Numbers (2 of 2)

- **randrange() function:** similar to `range()` function, but returns randomly selected integer from the resulting sequence
 - Same arguments as for the `range()` function
- **random() function:** returns a random float in the range of 0.0 to 1.0
 - Does not have any arguments
- **uniform() function:** returns a random float but allows user to specify range

Pseudo-Random Numbers

Random number created by functions in `random` module are actually pseudo-random numbers.

- **Seed value:** initializes the formula that generates random numbers
 - Need to use different seeds in order to get different series of random numbers
 - By default uses system time for seed
 - Can use `random.seed()` function to specify desired seed value



Writing Your Own Value-Returning Functions

Value-Returning Function

A value-returning function has a `return` statement that returns a value back to the caller of the function.

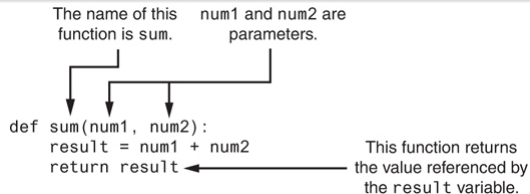


Figure 3: Parts of a function (Gaddis, 2021, pg.283)

- To write a value-returning function, you write a simple function and add one or more `return` statements
 - Format: `return expression`
 - The value for `expression` will be returned to the caller of your function
 - The the expression in the `return` statement can be a complex expression, such as a sum of two variables or the result of another value-returning function



Advantages of Value-Returning Functions

- Value-returning functions have the same benefits of functions we have discussed
 - simplify code
 - reduce duplication
 - provide for code reuse
- A major advantage of functions is increased ability to test code
 - A value-returning function is a black box: “data in, result out”
 - Can create tests of usual and edge cases for functions, to determine they work correctly and as expected
 - Tests are valuable products of software engineering systems, they give confidence code is working, and still working as it is modified



Using IPO Charts

- **IPO chart:** describes the input, processing and output of a function
 - Tool for designing and documenting functions
 - Typically laid out in columns
 - Usually provide brief descriptions of input, processing, and output, without going into details
 - Often includes enough information to be used instead of flowchart

The get_regular_price Function		
Input	Processing	Output
None	Prompts the user to enter an item's regular price	The item's regular price

The discount Function		
Input	Processing	Output
An item's regular price	Calculates an item's discount by multiplying the regular price by the global constant DISCOUNT_PERCENTAGE	The item's discount

Figure 4: IPO charts for the `get_regular_price()` and `discount()` functions (Gaddis, [2021](#), pg.283)



Tests and Python Docstring documentation

```
import math
import pytest

# The sum function accepts two numeric arguments and
# returns the sum of those arguments.
def area_of_circle(radius):
    """
    Given the radius of some circle, calculate and
    return the area of the circle.

    Parameters
    -----
    radius : float
        The radius of a circle we are being asked
        to calculate the area for.

    Returns
    -----
    area : float
        The area of the indicated circle is
        calculated and returned as the result of
        this function.
    """
    area = math.pi * radius**2.0
    return area
```

- Python Docstrings, many styles same as IPO but self-documenting code [Python Docstrings](#) (GeeksforGeeks, 2025)
- [pytest Tutorial: Effective Python Testing](#) (Hillard, 2024)

```
# example of pytest tests
def test_zero():
    area = area_of_circle(0.0)
    assert area == pytest.approx(0.0)

def test_unit_circle():
    area = area_of_circle(1.0)
    assert area == pytest.approx(math.pi)

def test_small_circle():
    area = area_of_circle(0.5)
    assert area == pytest.approx(0.7853981633974483)
```



Returning Boolean Values

- **Boolean function:** returns either True or False

- Use to test a condition such as for decision and repetition structures
 - Common calculations, such as whether a number is even, can be easily repeated by calling a function
- Use to simplify complex input validation code

```
def is_invalid(model_num):  
    if model_num != 100 and model_num != 200 and model_num != 300:  
        status = True  
    else:  
        status = False  
    return status  
  
# Get the model number.  
model = int(input('Enter the model number: '))  
# Validate the model number.  
while is_invalid(model):  
    print('The valid model numbers are 100, 200 and 300.')  
    model = int(input('Enter a valid model number: '))
```



Returning Multiple Values

- In Python, a function can return multiple values
 - Specified after the return statement separated by commas
 - Format: return expression1, expression2, etc.
- When you call such a function in an assignment statement, you need a separate variable on the left side of the = operator to receive each returned value

```
def get_name():  
    # Get the user's first and last names.  
    first = input('Enter your first name: ')  
    last = input('Enter your last name: ')  
    # Return both names.  
    return first, last  
  
first_name, last_name = get_name()
```



The math Module

- **math module:** part of standard library that contains functions that are useful for performing mathematical calculations
 - Typically accept one or more values as arguments, perform mathematical operation, and return the result
 - Use of module requires an `import math` statement
- The `math` module defines variables `pi` and `e` which are assigned the mathematical values for π and e
 - Can be used in equations that require these values, to get more accurate results
- Variables must also be referred to using the dot notation
 - Example: `circle_area = math.pi * radius**2.0`

Python math Module

The Python standard library's `math` module contains numerous functions that can be used in mathematical calculations.



The math Module

Table 1: Many of the functions in the `math` module

math Module Function	Description
<code>acos(x)</code>	Returns the arc cosine of <code>x</code> in radians
<code>asin(x)</code>	Returns the arc sine of <code>x</code> in radians
<code>atan(x)</code>	Returns the arc tangent of <code>x</code> in radians
<code>ceil(x)</code>	Returns the smallest integer that is greater than or equal to <code>x</code>
<code>cos(x)</code>	Returns the cosine of <code>x</code> in radians
<code>degrees(x)</code>	Assuming <code>x</code> is an angle in radians, the function returns the angle converted to degrees
<code>exp(x)</code>	Returns e^x
<code>floor(x)</code>	Returns the largest integer that is less than or equal to <code>x</code>
<code>hypot(x, y)</code>	Returns the length of a hypotenuse that extends from (0, 0) to (x, y)
<code>log(x)</code>	Returns the natural logarithm of <code>x</code>
<code>log10(x)</code>	Returns the base-10 logarithm of <code>x</code>
<code>radians(x)</code>	Assuming <code>x</code> is an angle in degrees, the function returns the angle converted to radians
<code>sin(x)</code>	Returns the sine of <code>x</code> in radians
<code>sqrt(x)</code>	Returns the square root of <code>x</code>
<code>tan(x)</code>	Returns the tangent of <code>x</code> in radians



Storing Functions in Modules

Python Modules

A module is a file that contains Python code. Large programs are easier to debug and maintain when they are divided into modules.

A module defines a **namespace** within which all functions and variables declared in the module reside and are accessed from.

- In large, complex programs, it is important to keep code organized
- **Modularization:** grouping related functions in modules
 - Makes program easier to understand, test and maintain
 - Make it easier to reuse code from multiple different programs
 - Import the module containing the required function to each program that needs it
- Modules contain function definitions but does not contain calls to the functions
 - Importing programs will call and use the functions in a module
- Rules for module names:
 - File name should end in `.py`
 - Cannot be the same as a Python keyword
- Import module using `import` statement



Summary

- **Value-returning functions** take input, perform a calculation, and return the result of the calculation
- Python contains a large and useful **standard library** that can be imported and used in your programs
 - When you `import` a module, it will be in a **namespace** defined by the module name
- You can write your own value-returning functions using the `return` keyword to return a value or expression
- Advantages of functions:
 - simplify code
 - reduce duplication
 - provide for code reuse
 - increase ability to test code
- **Modularization** is the grouping of related functions in modules, the standard library is a collection of useful modules that you can use



References

Gaddis, T. (2021). *Starting out with python* (5th). Pearson.

GeeksforGeeks. (2025). *Python docstrings*. Retrieved September 19, 2025, from <https://www.geeksforgeeks.org/python/python-docstrings/>

Hillard, D. (2024). *Pytest tutorial: Effective python testing*. Retrieved December 8, 2024, from <https://realpython.com/pytest-python-testing/>



EAST TEXAS A&M